SOFTWARE DESIGN DOCUMENT

Skeptic Engine

A Multi-Agent AI Platform for Audit Investigation & Fraud Detection

An adversarial debate framework that orchestrates six specialized AI agents to investigate financial transactions, with human-in-the-loop review and an immutable, hash-chained audit trail.

Next.js 15 · React 19 | FastAPI | LangGraph | Claude (Sonnet + Haiku) | PostgreSQL | Redis + Celery | WebSockets | Docker / K8s

Team
Athithiyan A · Aravinth Jay

Project
Capstone — v0.1.0

Document type
Engineering Design (SDD)

Date
29 June 2026

Status
Draft for Review

Contents

1. Introduction

This document describes the engineering design of Skeptic Engine: its architecture, components, data model, interfaces, and the design decisions behind them.

1.1 Purpose

Manual audit and fraud-review processes do not scale: a human reviewer can examine only a handful of flagged transactions per day, and single-pass automated checks either over-flag (alert fatigue) or miss context-dependent fraud. Skeptic Engine addresses this by automating the *investigation* of flagged transactions — not just their detection — while keeping a human accountable for every consequential decision. This document is the authoritative technical reference for engineers building, extending, or reviewing the system.

1.2 Scope

The design covers the end-to-end platform: the Next.js operator console, the FastAPI service layer, the LangGraph multi-agent orchestration core, the persistence and audit layers, and the real-time / asynchronous execution infrastructure. It documents the system as designed at v0.1.0, in which the frontend and backend are fully scaffolded and the agent crew, REST/WebSocket APIs, data model, and audit chain are implemented, with selected integrations (live external data feeds, production SSO) noted as forthcoming.

1.3 Intended Audience

Backend and frontend engineers extending the platform; reviewers and graders assessing the system design; and technical stakeholders evaluating the architecture for audit-compliance suitability. Readers are assumed to be comfortable with web architecture, REST, and basic LLM concepts.

1.4 Design Goals & Non-Goals

Goals	Non-Goals
Reduce hallucination via adversarial debate + claim verification	Fully autonomous decisions without human sign-off on risk
Full traceability — every decision tied to evidence and an immutable log	Real-time market/payments execution
Cost efficiency — auto-clear low-risk cases, reserve humans for high-risk	A general-purpose chatbot interface
Backend-swappable, horizontally scalable services	On-device / offline inference
Operator transparency — visible reasoning, citations, replay	Replacing the auditor's professional judgement

2. System Overview

Skeptic Engine ingests general-ledger transactions, flags risky ones with deterministic rules, then runs each flagged case through an AI "crew" that argues both sides before a human signs off.

2.1 The Core Idea: Professional Skepticism, Automated

Auditors are trained in *professional skepticism* — actively questioning whether a transaction is what it appears to be. A single LLM prompt tends to anchor on its first interpretation. Skeptic Engine encodes skepticism structurally: a **Challenger** agent argues the worst-case fraud interpretation, a **Defender** agent argues the legitimate-business rationale, and an **Adjudicator** weighs the debate. A **Verifier** then gates the conclusion, checking that each claim is grounded in collected evidence. This adversarial structure surfaces risk that one-shot scoring misses and produces a defensible reasoning trail.

2.2 Key Characteristics

Adversarial Debate

Challenger vs. Defender across bounded rounds before adjudication.

Human-in-the-Loop

High-risk and low-confidence cases require manual approval.

Immutable Audit

SHA-256 hash-chained, append-only event log of every action.

Grounded Claims

Verifier ties every assertion to a cited evidence artifact.

Real-Time

WebSocket streaming of agent progress to the operator console.

Confidence Gating

Configurable thresholds route cases to auto-clear or review.

2.3 Primary Actors

Actor	Responsibility
Audit Analyst	Uploads ledgers, monitors investigations, reviews the queue, approves/rejects/escalates.
Reviewer / Approver	Final human sign-off on high-risk cases; captures decision & rationale.
Administrator	Configures thresholds, models, roles, retention, knowledge-base sources.
Agent Crew	Autonomous AI services performing evidence, debate, adjudication, verification.

3. Architecture

A layered, service-oriented architecture: a React console talks to a FastAPI gateway, which persists state in PostgreSQL, dispatches long-running investigations to Celery workers, and streams progress back over WebSockets via a Redis bus.

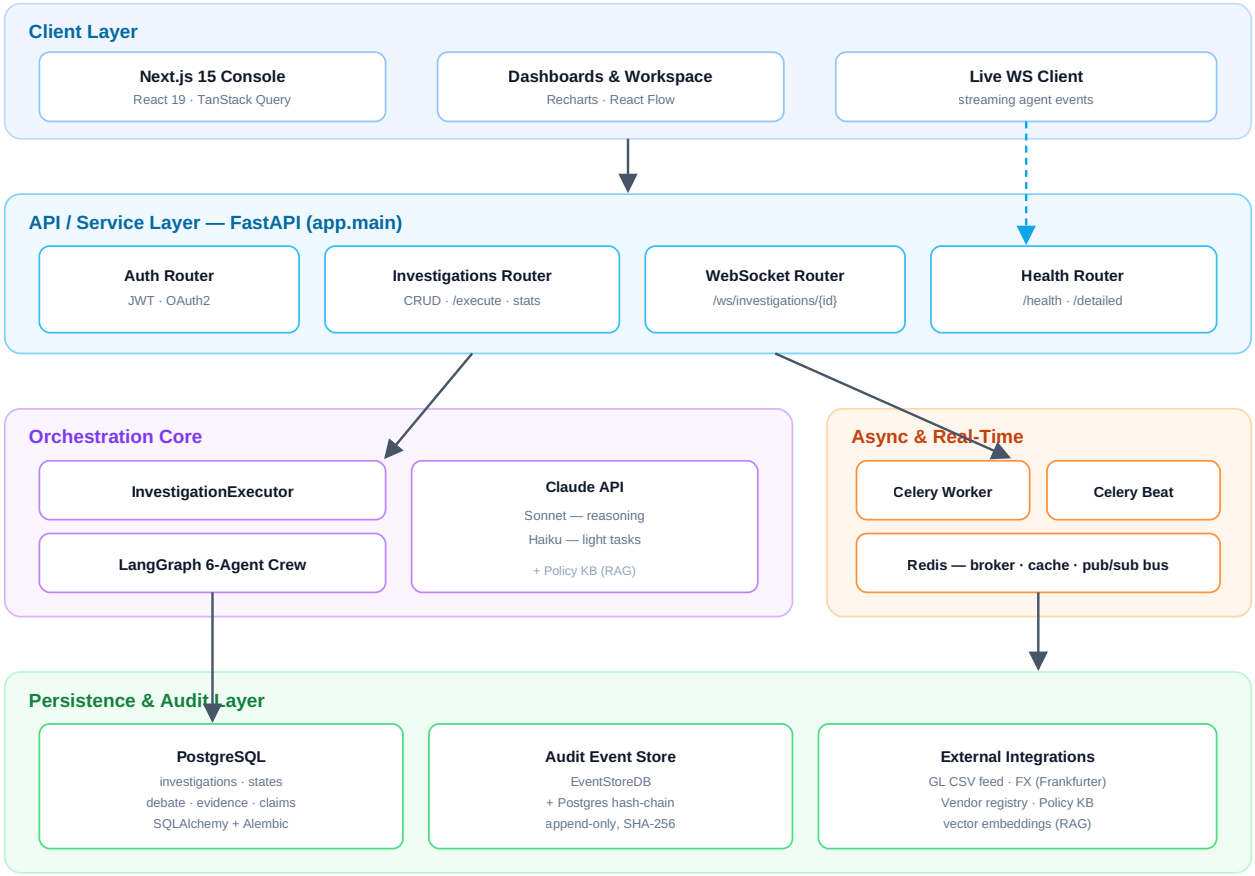

Figure 3.1 — High-level system architecture (dashed blue = real-time event stream).

3.1 Architectural Principles

- **Separation of concerns by layer.** Routers handle HTTP/WS only; orchestration logic lives in the agent/executor package; data access is isolated behind SQLAlchemy sessions. Each layer can be tested and replaced independently.
- **Domain-grouped packaging.** The backend is an installable `app/` package with subpackages (`core`, `api`, `agents`, `realtime`, `tasks`, `audit`, `db`, `schemas`) rather than flat modules — clarifying ownership and import boundaries.
- **Graceful degradation.** If no Celery broker is available, the API runs investigations in-process as a FastAPI background task; if Redis events are disabled, the WebSocket falls back to an in-process connection manager. The platform works end-to-end on a laptop.
- **State as the contract.** A single typed `InvestigationState` object flows through every agent and is checkpointed, making runs resumable, inspectable, and replayable.

3.2 Technology Stack

Layer	Technology	Rationale
Frontend	Next.js 15 (App Router), React 19, TypeScript (strict)	File-based routing, RSC, strong typing; future API routes available.
UI / State	TanStack Query & Table, React Hook Form, Zod, Tailwind, Radix	Decoupled server state, accessible primitives, schema-validated forms.
Visualization	Recharts, React Flow, Lucide	Charts for analytics; DAG for workflow replay/debug.
API	FastAPI, Pydantic, Uvicorn	Async, auto-typed schemas, OpenAPI, native WebSocket support.
Orchestration	LangGraph, LangChain-Anthropic	Stateful graph execution with explicit nodes/transitions for agents.
LLM	Claude 3.5 Sonnet + Haiku	Sonnet for reasoning/debate; Haiku for cheap, light tasks.
Data	PostgreSQL, SQLAlchemy, Alembic	Relational integrity, JSON columns for flexible state, versioned migrations.
Async / RT	Celery, Redis, WebSockets	Offload long LLM runs; pub/sub bridge for cross-process streaming.
Audit	EventStoreDB (+ Postgres fallback)	Append-only event log with cryptographic hash chaining.
Infra	Docker Compose, Kubernetes	Reproducible local stack; container-native production deploy.

4. Multi-Agent Crew Design

Six specialized agents, orchestrated as a LangGraph state machine, each with a narrow responsibility and a distinct prompt persona. A Supervisor routes control between phases.

4.1 The Agents

Agent	Model	Responsibility
Supervisor	Sonnet	Orchestrates the workflow — decides the next phase, manages debate-round counting, and routes to specialists. Implemented as the graph's routing node.
Evidence	Sonnet	Performs RAG over the policy knowledge base and pulls external context (FX rates, vendor registry, history), emitting evidence artifacts with citations and relevance scores.
Challenger	Sonnet	Argues the strongest fraud / control-failure interpretation. Surfaces red flags from the evidence.
Defender	Sonnet	Argues the legitimate-business rationale. Provides the steel-man counter-case.
Adjudicator	Sonnet	Weighs both sides of the transcript and produces a risk verdict and a confidence score.
Verifier	Sonnet	QA gate — checks each material claim is grounded in a cited evidence artifact; flags unsupported assertions (hallucination control).

4.2 Orchestration State Machine

The Supervisor implements explicit transitions over a workflow state. Debate loops back to the Supervisor each round; once `debate_round ≥ max_debate_rounds` (default 2), control advances to adjudication, then verification.

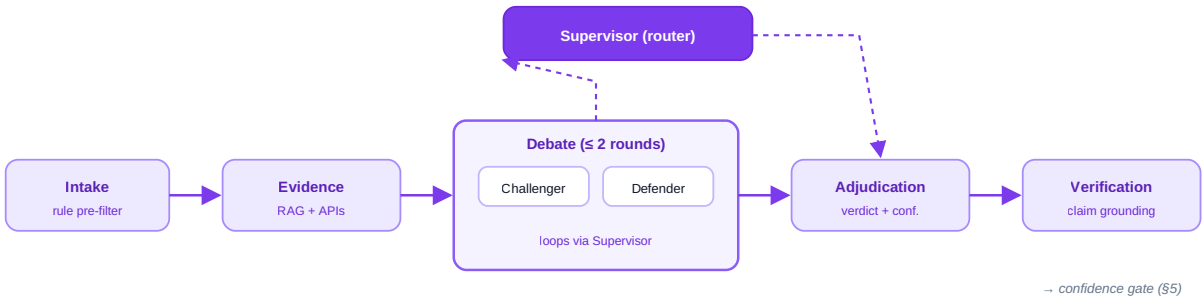

Figure 4.1 — Agent orchestration state machine. Solid = phase flow; dashed = Supervisor routing.

4.3 Shared State Object

All agents read and write a single `InvestigationState` (a LangGraph `TypedDict`). This is the integration contract between agents and the persistence layer.

```
class InvestigationState(TypedDict, total=False):
    investigation_id: str
    transaction_id: str; vendor: str; category: str; amount: float
    materiality: float
    evidence: list[dict]            # artifacts + citations
    evidence_summary: str
    debate_round: int; max_debate_rounds: int
    challenger_arguments: list[str]
    defender_arguments: list[str]
    debate_transcript: list[InvestigationMessage]
    adjudication: dict              # verdict + confidence
    verification_results: dict      # grounded? per claim
    messages: list[InvestigationMessage]
    workflow_state: str; status: str
```

4.4 Resilience & Cost Controls

- **Lazy LLM imports.** `langgraph` and `langchain_anthropic` are imported lazily so the module loads (and unit tests run with stub agents) without the heavy dependencies or an API key.
- **Robust JSON parsing.** Agent outputs are parsed defensively (`_extract_json` falls back to a regex extraction) to tolerate prose-wrapped model responses.
- **Bounded debate.** `MAX_DEBATE_ROUNDS` caps token spend and guarantees termination.
- **Model tiering.** Sonnet drives reasoning; Haiku is reserved for lightweight generation, reducing cost per case.

5. Investigation Pipeline & Decision Logic

Each case moves through a defined status lifecycle, ending in an automated or human-gated decision driven by configurable confidence thresholds.

5.1 Status Lifecycle

The `InvestigationStatus` enum defines the pipeline stages:

`INTAKE` → `COLLECTING_EVIDENCE` → `AGENT_DEBATE` → `VERIFICATION` → `HUMAN_REVIEW` → `REPORT_READY` → `CLOSED` (`FAILED` on error)

5.2 The Five Phases

Phase	Name	What happens
0	Case Intake	CSV ledger uploaded → transactions normalized → deterministic rule pre-filter flags candidates and assigns initial risk/materiality.
1	Evidence Collection	Supervisor decomposes the case; Evidence agent runs RAG over policy KB + live APIs, producing cited artifacts and a summary.
2	Debate + Verification	Challenger ↔ Defender across ≤2 rounds; Adjudicator issues a verdict + confidence; Verifier QA-gates claim grounding.
3	Decision + Human-in-Loop	Confidence gate routes the case (auto-clear / manual review / escalation). High-risk always reaches a human.
4	Report + Audit	Report artifact generated (MD/HTML/PDF); immutable audit entries appended; case closed.

5.3 Confidence Gate

Routing is driven by configurable thresholds (`DEFAULT_CONFIDENCE_THRESHOLD = 0.70` , `DEFAULT_ESCALATION_THRESHOLD = 0.50`). The representative policy:

Condition	Route	Rationale
confidence ≥ 0.90 *and* verified *and* low risk	AUTO-CLEAR	High certainty, no material risk — no human needed.
0.70 ≤ confidence < 0.90	MANUAL REVIEW	Plausible but not certain — routed to the review queue.
confidence < 0.70, or any high/ critical risk	ESCALATED REVIEW	Low certainty or material exposure — senior sign-off required.

> **DESIGN PRINCIPLE**
> Thresholds are externalized as settings, not hard-coded in agent logic. An administrator can tune the platform's risk appetite without redeploying agents.

5.4 Risk Taxonomy

CRITICAL HIGH MEDIUM LOW CLEARED

6. Data Model

A normalized relational schema centered on the `investigations` table, with child tables for each artifact type and a separate append-only audit log.

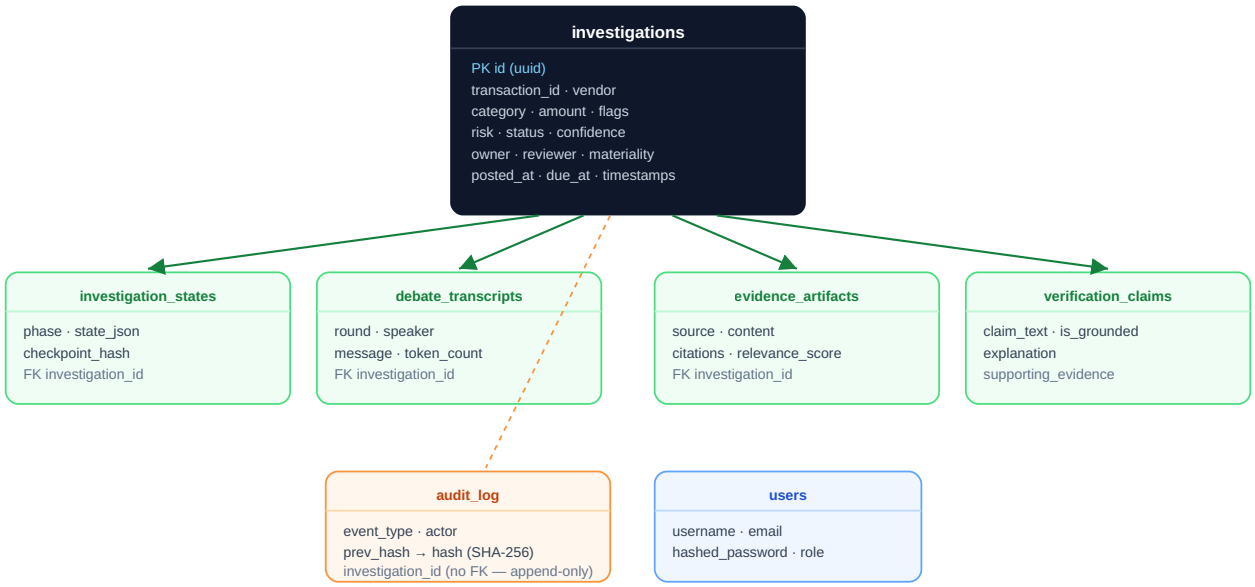

Figure 6.1 — Entity-relationship overview. Green = 1-to-many child artifacts (cascade delete); orange = independent append-only log.

6.1 Core Entities

Table	Purpose & notable columns
`investigations`	The case aggregate. Holds transaction facts, AI outputs (`confidence` , `risk`), workflow `status` , ownership, and SLA dates (`due_at` = posted + 7 days). Indexed on `transaction_id` , `risk` , `status` , and a composite `(transaction_id, vendor)` .
`investigation_states`	LangGraph checkpoints — `phase` , full `state_json` , and a `checkpoint_hash` . Enables resume and step-by-step replay.
`debate_transcripts`	One row per utterance: `round` , `speaker` , `message` , `token_count` . Drives the debate viewer.
`evidence_artifacts`	Collected evidence with `source` , `content` , `citations` (JSON), and a `relevance_score` .
`verification_claims`	Per-claim grounding: `claim_text` , `is_grounded` , `explanation` , `supporting_evidence` .
`audit_log`	Append-only events with `event_type` , `actor` , and a SHA-256 hash chain. Intentionally has no FK so records survive case deletion.
`users`	Auth identities — `username` , hashed password, and `role` for RBAC.

6.2 Design Notes

- **UUID primary keys** avoid cross-system collisions and don't leak volume.
- **JSON columns** (`flags` , `citations` , `state_json`) keep semi-structured agent output flexible without schema churn, while core query fields stay typed and indexed.
- **Cascade deletes** on child artifacts keep the relational graph clean; the audit log is deliberately excluded so compliance history is never removed.
- **Migrations** are managed by Alembic, with `env.py` wired to the ORM `Base` for autogeneration.

7. API Design

A versioned REST API under `/api/v1` plus a WebSocket channel. All data routes require a JWT bearer token; schemas are Pydantic models with auto-generated OpenAPI docs.

7.1 REST Endpoints

Method	Path	Description
POST	`/api/v1/auth/token`	Exchange username/password for a JWT (OAuth2 password flow).
POST	`/api/v1/auth/register`	Create a user (unique username/email enforced).
POST	`/api/v1/investigations`	Create a case; writes a `case_created` audit event.
GET	`/api/v1/investigations`	List with pagination (`skip` / `limit` , capped at 500) and `status` / `risk` filters.
GET	`/api/v1/investigations/{id}`	Fetch a single case (404 if missing).
GET	`/api/v1/investigations/stats/summary`	Dashboard aggregates: totals, avg confidence, counts by risk and status.
GET	`/api/v1/investigations/{id}/debate`	Ordered debate transcript.
GET	`/api/v1/investigations/{id}/evidence`	Evidence artifacts for the case.
GET	`/api/v1/investigations/{id}/verification`	Verification claim records.
GET	`/api/v1/investigations/{id}/audit`	Immutable audit trail (from the event store).
POST	`/api/v1/investigations/{id}/execute`	Kick off the agent crew — via Celery, or in-process when `USE_CELERY=false` .
GET	`/health` , `/health/detailed`	Liveness/readiness probes.

7.2 WebSocket Channel

`WS /api/v1/ws/investigations/{id}` streams real-time agent events. On connect, the server registers the socket with the connection manager and spawns a task that forwards Redis pub/sub events for that case; client messages are acknowledged. On disconnect, the forwarder is cancelled and the socket deregistered.

7.3 Representative Payloads

```
# Create
POST /api/v1/investigations
{ "transaction_id":"GL-88213", "vendor":"Acme Holdings",
  "category":"Consulting", "amount":128400.00,
  "materiality":50000, "owner":"a.nair" }

# Execute → 200
{ "investigation_id":"…", "task_id":"…",
  "status":"running", "message":"Investigation queued" }

# WS event frame
{ "type":"agent_update", "agent":"challenger",
  "phase":"debate", "round":1, "message":"…" }
```

7.4 Conventions

- **Auth.** Every data route depends on `get_current_user`; tokens carry the user's role for downstream RBAC.
- **Validation.** Request/response bodies are Pydantic schemas (`InvestigationCreate`, `InvestigationOut`, …); invalid input yields a 422 automatically.
- **Pagination & safety.** List limits are clamped server-side; missing resources return 404, not empty 200s.

8. Key Sequence Flows

The canonical create → execute → stream → decide flow, showing how the API, async worker, agent crew, datastore, and real-time bus interact.

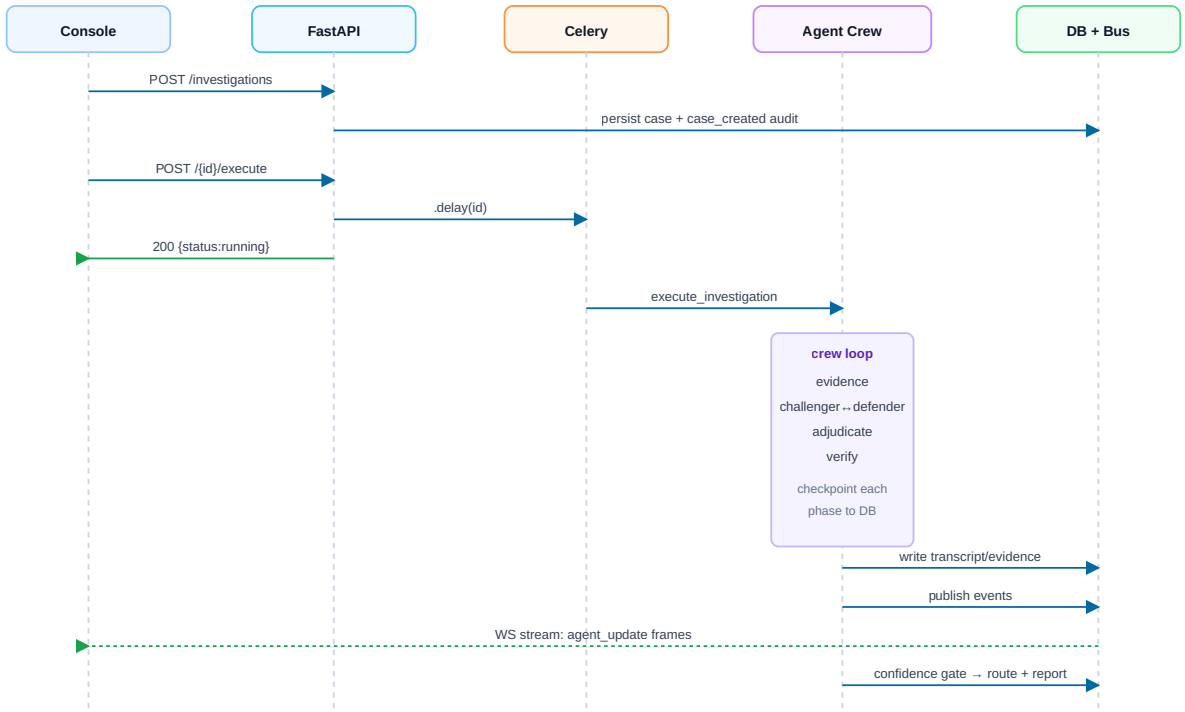

Figure 8.1 — Create → execute → stream → decide sequence (blue = request, green = response/stream).

When `USE_CELERY=false`, the `.delay()` hop is replaced by a FastAPI `BackgroundTask` running `_run_investigation_inline` with its own DB session — the rest of the flow is identical, which is what makes single-machine demos and CI runs faithful to production.

9. Real-Time & Asynchronous Execution

LLM investigations take seconds to minutes, so they run off the request path. Progress is streamed back to operators through a Redis-backed pub/sub bridge.

9.1 Why Asynchronous

An investigation invokes the Claude API many times across evidence, debate, adjudication, and verification. Blocking an HTTP request for the full run would exhaust connections and time out. The `/execute` endpoint therefore returns immediately with a `running` status and a task handle, while the work proceeds in a Celery worker.

9.2 Task Topology

Process	Command	Role
API server	`uvicorn app.main:app`	HTTP + WebSocket gateway.
Celery worker	`celery -A app.tasks.celery_app worker`	Runs `InvestigationExecutor` per case.
Celery beat	`celery -A app.tasks.celery_app beat`	Scheduled/periodic jobs.
Redis	broker · result backend · cache · pub/sub	Decouples API ↔ worker; carries live events (separate logical DBs).

9.3 Event Streaming Bridge

As the crew progresses, the executor publishes events to a per-case Redis channel (`investigation_events:{id}`). The WebSocket route subscribes to that channel and forwards each event to connected clients. This **cross-process bridge** matters because the worker producing events and the API holding the socket are different processes — Redis is the seam that connects them. In local mode (`USE_REDIS_EVENTS=false`), an in-process connection manager serves the same role.

> **RESILIENCE**
> The Redis socket timeout is deliberately short (0.25 s) so a stalled bus degrades the live feed without blocking the investigation itself — the case still completes and persists.

10. Audit & Compliance

Auditability is a first-class requirement, not a log file. Every consequential action is recorded in an append-only, cryptographically chained event store.

10.1 Hash-Chained Event Log

Each audit entry stores the SHA-256 hash of its own content together with the previous entry's hash, forming a tamper-evident chain: altering or deleting any historical record breaks every subsequent hash. The primary backend is EventStoreDB, with a Postgres hash-chain implementation as a fallback so the guarantee holds even without the dedicated event store.

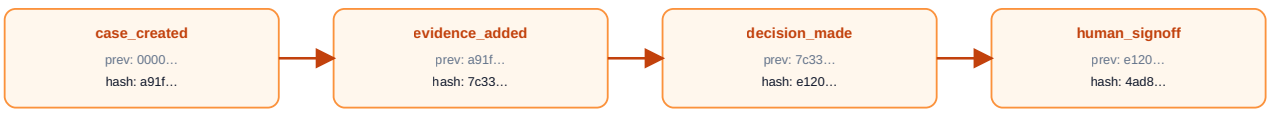

Figure 10.1 — Each event embeds the prior event's hash, making the trail tamper-evident.

10.2 Compliance Properties

- **Traceability.** Every decision links to the evidence, debate transcript, and verification that produced it.

- **Segregation of duties.** Role-bearing tokens separate who creates, reviews, and approves a case.
- **Human accountability.** Sign-off events capture the approving actor and rationale for high-risk decisions.
- **Retention.** The audit log has no foreign key to cases, so history survives operational data deletion.

11. Frontend Design

A Next.js App Router console organized by feature, with a clean service → hook → feature → page layering and mock-to-real backend swapability.

11.1 Layered Structure

Layer	Responsibility
`types/`	The TypeScript domain model — single source of truth, mirroring backend schemas.
`services/`	Pure data-access functions returning typed data. The seam where mock data is swapped for real `fetch` calls.
`hooks/`	TanStack Query wrappers (one per domain) providing caching, refetch, and polling.
`features/`	Page-level views composing hooks + components and handling loading/error states.
`components/`	Reusable UI, domain (EvidenceCard, DebateMessage), layout (AppShell), and chart/table components.
`app/`	Next.js routes wiring features to URLs.

> **WHY THIS MATTERS**
>
> Because the UI talks only to the service layer, the entire console runs today on typed mock fixtures and switches to the live API by changing `services/` alone — no component rewrites.

11.2 Key Screens

Route	Purpose
`/dashboard`	Executive overview — metrics, risk distribution, agent health, case trends, API cost.
`/intake`	CSV upload and rule pre-filter configuration.
`/investigations` · `/[caseId]`	Case list with filters; per-case workspace (details, pipeline, evidence, debate, verification, review actions, audit trail).
`/debate`	Challenger ↔ Defender transcript viewer.
`/evidence` · `/verification`	Evidence explorer and claim-grounding QA panel.
`/review`	Human review queue — sortable, bulk actions, signature capture.
`/reports` · `/audit-logs`	Generated reports and the immutable transaction history.
`/replay`	Step-by-step investigation playback over the React Flow DAG.
`/analytics` · `/evaluation`	Confidence/verifier/hallucination trends; RAGAS scorecards (faithfulness, context precision/recall, agent goal accuracy) tracked per release.
`/knowledge-base` · `/settings`	Policy sources & embeddings; governance (thresholds, models, SoD, retention).

12. Deployment & Infrastructure

Container-native: a single Docker Compose stack for local development and Kubernetes manifests for production, with the same image running the API, worker, and beat processes.

12.1 Topology

Concern	Approach
Containerization	Single `Dockerfile` (CMD `uvicorn app.main:app`); the same image runs API, Celery worker, and beat with different commands.
Local stack	`docker-compose.yml` brings up api / worker / beat / flower plus Postgres and Redis; a separate compose file provisions local infra only.
Orchestration	`k8s-deployment.yaml` for production — independently scalable API and worker deployments.
Migrations	Alembic (`alembic upgrade head`) on deploy.
Observability	Flower for Celery; `/health` probes for liveness/readiness.
Frontend	Next.js — Vercel-native, or static export behind a CDN.

12.2 Configuration Management

All runtime configuration is centralized in a `pydantic-settings` `Settings` object reading from the environment, with sane defaults and a `.env.example` template. A model validator enforces invariants — e.g. an Anthropic API key is required when `USE_REAL_AGENTS=true`. Key knobs: database pool sizing, Redis channels/TTL, Celery broker/result URLs, Claude model names, and the materiality / confidence / escalation thresholds.

> **SECURITY NOTE**
> Secrets (the Anthropic API key, DB credentials) must be injected from the environment or a secret manager and never committed. Rotate any key that has appeared in source control, and keep `.env` out of version control.

13. Security

13.1 Authentication & Authorization

- **Authentication.** OAuth2 password flow issues JWTs; passwords are hashed (never stored in plaintext) via the security module.
- **Authorization.** Tokens encode a `role`, enabling role-based access control and segregation of duties across analyst / reviewer / admin functions.
- **Route protection.** Every data route depends on `get_current_user`; unauthenticated requests are rejected at the dependency layer.

13.2 Data & Operational Security

- **Tamper-evidence.** The hash-chained audit log detects any post-hoc modification of history.
- **Input validation.** Pydantic schemas reject malformed payloads before they reach business logic.
- **Secrets hygiene.** Configuration is environment-driven; production secrets belong in a secret manager, with regular rotation.
- **Least privilege (target).** Database roles and per-service credentials should be scoped to only the access each process needs.

13.3 Known Hardening Backlog

Production SSO (OIDC/Okta), fine-grained RBAC enforcement per endpoint, rate limiting, and secrets-manager integration are tracked as forthcoming work (§16).

14. Testing & Evaluation Strategy

Two complementary layers: deterministic unit/integration tests that prove the system *runs correctly*, and a RAGAS evaluation harness that proves the agent crew *reasons correctly*.

14.1 Functional Test Suites

Tests run with no external services: an in-memory/SQLite database and stub agents stand in for Postgres and the Claude API, keeping the suite fast and deterministic.

Suite	Coverage
`test_api.py`	Investigation CRUD, listing/filtering, stats aggregation, 404 handling.
`test_auth.py`	Registration, login, token issuance, protected-route rejection.
`test_agents.py`	Crew state machine transitions and debate-round termination (stub LLM).
`test_executor.py`	End-to-end executor pipeline and state checkpointing.
`test_audit.py`	Hash-chain integrity — appends link correctly and tampering is detectable.

Backend gate

`pytest` — 14 tests, no external dependencies.

Frontend gate

`lint · typecheck · test · build` enforced by Husky pre-commit.

Determinism

Stub agents make AI-dependent paths reproducible in CI.

14.2 RAGAS Evaluation Framework

Functional tests cannot tell us whether the crew's *verdicts* are well-grounded and correct. For that, Skeptic Engine adopts **RAGAS** (Retrieval-Augmented Generation Assessment) — an LLM-assisted evaluation framework purpose-built for RAG and agentic pipelines. Because every investigation is, structurally, a retrieval-augmented, multi-step agentic generation, RAGAS maps cleanly onto the platform: retrieval metrics score the Evidence agent, generation metrics score the Adjudicator's verdict, and agentic metrics score the crew as a whole.

Evaluation runs against a curated **golden set** of 30–50 labelled cases drawn from the synthetic GL data, each with a ground-truth verdict, a reference answer, and the policy clauses that *should* be retrieved. Scores are computed per release, regressions block merges, and trends surface on the `/evaluation` screen.

Retrieval quality — the Evidence agent

RAGAS metric	What it measures here
Context Precision	Of the policy/context chunks the Evidence agent retrieved, what fraction are actually relevant to the case — penalises noisy retrieval.
Context Recall	Of the information needed to reach the correct verdict, how much was present in the retrieved evidence — catches missed policy clauses.
Context Entity Recall	Coverage of key entities (vendor, amount, GL account, related parties) in the retrieved context — vital for related-party detection.
Noise Sensitivity	How often irrelevant retrieved chunks lead to incorrect or unsupported claims — measures robustness of the downstream agents to noisy evidence.

Generation quality — the verdict

RAGAS metric	What it measures here
Faithfulness	Proportion of claims in the adjudication that are grounded in the retrieved evidence. This is the quantitative twin of the Verifier's grounding gate — the platform's primary hallucination metric.
Response (Answer) Relevancy	Whether the verdict actually addresses the case question rather than drifting into generic commentary.
Factual Correctness	Agreement between the crew's verdict and the labelled ground-truth (claim-level precision/recall against the reference).
Semantic Similarity	Embedding-level closeness between the generated verdict/rationale and the reference answer.

Agentic quality — the crew

RAGAS metric	What it measures here
Tool-Call Accuracy / Tool-Call F1	Whether the Evidence agent invokes the right tools (FX, vendor registry, policy KB) with the right arguments, in the right order.
Topic Adherence	Whether Challenger and Defender stay within audit scope instead of speculating off-topic.
Agent Goal Accuracy	Whether the crew achieves the end-to-end investigation goal — a correct, routed, fully-supported decision.

> **WHY RAGAS REPLACES THE OLD A/B HARNESS**
>
> The earlier plan compared the crew against a single-prompt baseline on ad-hoc false-positive and hallucination counts. RAGAS gives reference-backed, reproducible, per-component scores instead: **Faithfulness** and **Context Recall** pinpoint *where* a regression occurred (retrieval vs. reasoning), not just that quality dropped — and the same golden set still supports a crew-vs-baseline comparison when needed.

14.3 Quality Gates

Gate	Criteria
Backend	`pytest` — 14 tests, no external dependencies.
Frontend	`lint · typecheck · test · build` enforced by Husky pre-commit.
Evaluation	RAGAS suite on the golden set meets minimum thresholds (e.g. Faithfulness ≥ 0.90, Context Recall ≥ 0.85, Agent Goal Accuracy ≥ 0.80); a drop blocks release.

Forthcoming: end-to-end browser tests (Playwright) and automated RAGAS runs wired into CI with score dashboards.

15. Design Decisions & Trade-offs

Decision	Why	Trade-off accepted
Multi-agent debate over single prompt	Surfaces opposing interpretations; reduces anchoring and hallucination; produces an auditable reasoning trail.	Higher token cost and latency per case (mitigated by bounded rounds and auto-clear).
LangGraph state machine	Explicit nodes/transitions make orchestration inspectable, resumable, and replayable.	More upfront structure than a free-form agent loop.
Mandatory human-in-the-loop on risk	Compliance and accountability — AI assists, humans decide.	Throughput ceiling on high-risk volume; deliberate.
Async (Celery) execution</		